

Projeto Ambientes Separados

Youse Seguradora

Proposta para o CTO | Versão 1.3 | Junho 2026

Auditoria GitHub: 461 repos | 313 refs QA | 46 refs Stage

Projeto Youse: Estratégia de Ambientes Separados

Organização: Youse Seguradora

Versão: 1.3 | **Data:** Junho/2026

Status: Proposta para apresentação ao CTO e times

1. Introdução — por que este documento existe?

O que está acontecendo?

Hoje, na Youse, **quase todo mundo testa no mesmo lugar**: o ambiente QA (`qa.youse.io`). Desenvolvedores, analistas de QA, time de produto e automação de testes usam **o mesmo endereço**, ao mesmo tempo, com versões diferentes de projetos.

Isso gera situações que todos conhecem:

- QA vai testar e o ambiente **está quebrado** — porque outro time subiu algo antes
- Mensagens no Slack: *"parem de usar QA, vou mexer no ambiente"*
- Automação de testes **falha sem motivo** — alguém mudou toggle, regra de negócio ou dado
- Testes de **performance e caos no Stage** deixam o ambiente lento para quem precisa integrar
- Ninguém sabe com certeza **o que está rodando** em QA naquele momento

Não é culpa de uma pessoa ou de um time. É **falta de regra clara sobre onde cada atividade deve acontecer**.

O que este documento propõe?

Uma **estratégia única** para organizar os ambientes de teste e homologação da Youse — em linguagem acessível para dev, QA, produto e liderança.

Não são duas soluções concorrentes. São duas camadas que trabalham juntas:

Camada	Nome	Para quê?	Analogia do dia a dia
Camada 1	Ambientes fixos	Releases, regressão, validação de produto, pré-produção	Salas permanentes do prédio — cada uma com uma função
Camada 2	Ambiente temporário por branch	Desenvolvimento e teste de uma tarefa específica	Mesa de trabalho montada só para aquela entrega — desmontada quando acaba

Camada 1 — Ambientes fixos (a proposta inicial)

Ideia central: **cada ambiente tem um papel**. Ninguém usa o lugar errado.

Ambiente	Quem usa	Em uma frase
Stage	Integração após merge	"A linha de montagem — código integrado"
Automation	Robôs de teste (E2E)	"Laboratório fechado — ninguém mexe nas regras"
QA / UAT	QA manual + Produto	"Sala de prova — só entra release estável"
Pre-Prod	Último check antes de prod	"Ensaio geral"
Perf / Caos	Testes de carga e falhas	"Pista de stress — isolada do resto"
Produção	Clientes	"Show ao vivo"

Resolve: automação instável, performance no Stage, QA recebendo release candidate estável.

Camada 2 — Ambiente por branch (sua sugestão)

Ideia central: **cada tarefa (branch) ganha um ambiente próprio**, que **some quando a tarefa termina**.

- 1. Dev abre branch da tarefa → sistema cria URL exclusiva (ex.: `you-123.preview.youse.io`)
- 2. Dev desenvolve e testa **ali** — não no QA compartilhado
- 3. Dev avisa QA → QA testa **na mesma URL** (mesmo ambiente, mesma branch)
- 4. Aprovado → merge → segue para Stage, Pre-Prod, Produção
- 5. Tarefa encerrada → ambiente **destruído automaticamente**

Resolve: conflito no QA, mensagens "parem de usar QA", dev e QA testando coisas diferentes.

As duas camadas juntas — visão geral

DURANTE A TAREFA (dia a dia)	APÓS MERGE (release)
Dev + QA no preview da branch (ambiente temporário)	Stage → Automation → QA/UAT → Pre-Prod → Prod (ambientes fixos)
↓	↓
Destruído ao fechar tarefa	Caminho até produção

Resumindo: preview por branch cuida do **dia a dia**; ambientes fixos cuidam da **confiança no release**.

Evidências que reforçam a necessidade

Auditoria dos repositórios GitHub da Youse (461 projetos):

Fato	O que significa
313 referências a <code>qa.youse.io</code> no código	QA virou o ambiente padrão de tudo
46 referências a Stage	Stage é subutilizado para o volume de QA
Handbook define papéis diferentes	A intenção existe — a prática divergiu
<code>performance-tests</code> roda em Stage/QA	Carga destrutiva no meio de quem integra

Detalhes técnicos da auditoria estão no **Anexo A**.

2. Sumário Executivo

Este projeto propõe **reorganizar os ambientes da Youse** para que cada time saiba **onde testar**, **quando testar** e **sem atrapalhar os outros**.

Benefícios esperados:

- Menos tempo perdido esperando ambiente estável
- QA testa o que o dev entregou — não "o que sobrou no QA"
- Automação confiável, sem falsos alarmes
- Fim das mensagens reativas no Slack
- Releases mais previsíveis

Investimento: fases ao longo de 90 dias. A Fase 1 não exige infraestrutura nova — começa por regras e processo.

3. Situação Atual (AS-IS)

3.1 Ambientes que temos hoje

Ambiente	URL	Uso declarado (handbook)	Uso real hoje (auditoria + operação)
QA	https://qa.youse.io/	Testes em desenvolvimento e E2E pós-desenvolvimento	Todos (dev, QA, produto, automação, demos) — 313 refs no código
Stage	https://www-stage.youse.io/	Integração contínua, pacotes fechados com tags, E2E de jornadas	Integração + testes de performance (k6) + caos — 46 refs
Produção	https://www.youse.io/ / https://youse.com.br/	Clientes finais	Clientes finais

3.2 O que a documentação interna diz vs. o que acontece na prática

O handbook da Youse (`handbook/docs/v1/quality/ambientes.md`) já define papéis para QA e Stage. **O problema não é falta de documentação — é que a operação diária não segue o que está escrito.**

Ambiente	O handbook diz	O que acontece de verdade
QA	Testes em desenvolvimento e E2E	Todos usam: dev, QA, produto, automação
Stage	Estável, só pacotes aprovados	Integração + testes de performance
Produção	Clientes finais	OK

3.3 Detalhes da auditoria GitHub (referência técnica)

Escopo: org `youse-seguradora` , 461 repositórios, Jun/2026.

Panorama quantitativo

Categoria	Quantidade	Observação
Total de repositórios	461	Inclui ativos, arquivados e tooling
Referências <code>qa.youse.io</code>	313	Configs, specs, Terraform, Ansible, Helm, .env
Referências <code>www-stage.youse.io</code>	46	Deploy, integração, performance-tests
Referências <code>perf.youse.io</code>	0	Ambiente não existe no código
Referências <code>auto.youse.io</code>	0	Ambiente não existe no código
CircleCI (<code>.circleci/config.yml</code>)	~159	Pipeline principal de deploy
Jenkins (<code>Jenkinsfile</code>)	~117	Legado + scripts operacionais
GitHub Actions (<code>.github/workflows</code>)	~7	Crescimento recente (ex.: <code>qa-e2e-tests-automation</code>)

Repositórios e ferramentas (resumo)

Camada	Tecnologia	Repositórios relevantes (auditoria)
Backend / Core	Ruby on Rails, microserviços	<code>youse</code> (monolito em desmonte), <code>order-service</code> , <code>billing-cloud</code> , <code>inspection-service</code> , <code>notifier</code> , <code>bff</code> , <code>call-center-bff</code>
Frontend	React, TypeScript, Lerna monorepo	<code>sales-frontend</code> , <code>cargo-components</code> , <code>dashboard</code> , <code>partner-portal</code>
Mobile	Flutter	<code>mob-flutter</code> (<code>.envs/.env.qa.json</code>)
CI/CD	CircleCI Orb + Helm	<code>deployment-orb</code> , <code>youse</code> (<code>.helm/qa.yaml</code> , <code>.helm/stage.yaml</code>), <code>sales-frontend</code>
Automação E2E	Playwright, Cucumber (legado)	<code>qa-e2e-tests-automation</code> (Playwright + GitHub Actions), <code>automation-tests</code> (Cucumber), <code>platform-automated-testing</code>
Performance	k6	<code>performance-tests</code> — alterna QA/Stage via <code>__ENV.ENV</code>
Infra / IaC	Terraform, Ansible, Kubernetes	<code>terraform</code> , <code>terraform-platform</code> , <code>devops</code> , <code>gitops-kubernetes-addons</code>
Documentação	Handbook interno	<code>handbook</code> , <code>insomnia-document</code>
DevOps legado	Jenkins	<code>youse-jenkins-v2</code> , <code>jose-scripts</code>

2.2.4 Detalhamento dos repositórios críticos

Repositório	CI/CD	Deploy QA	Deploy Stage	Automação	Observação
youse	CircleCI	.helm/qa.yaml	.helm/stage.yaml	Specs com refs QA	Core app — maior volume de refs QA
sales-frontend	CircleCI	Pipeline	Pipeline	—	Monorepo React/TS
deployment-orb	CircleCI	.helm/qa.yaml	—	—	Orb compartilhado de deploy
qa-e2e-tests-automation	GitHub Actions	Target QA	—	Playwright	Suite E2E moderna
automation-tests	—	features/support/env.rb → QA	—	Cucumber (legado)	Desatualizado (2021)
platform-automated-testing	—	.env.qa	—	Testes plataforma	Topic: automated-tests
performance-tests	—	k6 → QA	k6 → Stage	k6 load/stress	Alterna ambientes — risco P10/P11
devops	Ansible	ansible_v2/vars/*/qa_envs.yml	—	—	Configs QA por serviço
terraform	Atlantis	services/*/qa/	—	—	Route53, infra QA
terraform-platform	Atlantis	qa/*/	—	—	Infra plataforma QA
handbook	—	—	—	—	Documentação oficial (desatualizada vs realidade)

2.3 Handbook vs. Realidade — gap analysis

Aspecto	Handbook (ambientes.md)	Realidade (auditoria + operação)	Gap
Papel do QA	Testes em desenvolvimento e E2E pós-dev	Ambiente universal: dev + QA + produto + automação + demos	Crítico — QA absorve todos os papéis
Papel do Stage	Estável, pacotes fechados, E2E de jornadas	Integração + performance k6 + caos	Alto — Stage sobrecarregado
Automação E2E	Stage executa E2E de jornadas	E2E roda em QA (qa-e2e-tests-automation , automation-tests) e Stage	Alto — sem ambiente dedicado
Performance	Não mencionado	performance-tests alterna QA/Stage via env var	Crítico — sem perf.youse.io
Ambiente Automation	Não existe	Não existe (0 refs auto.youse.io)	Crítico — proposta TO-BE
Ambiente Pre-Prod	Não existe	Não existe	Médio — proposta TO-BE
Volume de refs	QA e Stage equivalentes	QA 313 refs vs Stage 46 refs (6,8:1)	Crítico — confirma concentração em QA
Deploy manual	Não documentado	Deploy manual frequente em QA	Alto — sem gate de promoção
Feature flags	Não documentado	Toggles alterados manualmente em QA	Alto — sem governança por ambiente
Comunicação	Não documentado	Slack reativo ("parem de usar QA")	Médio — sem calendário

2.4 Fluxo atual (simplificado)

```
Desenvolvedor → deploy manual ou pipeline → QA (313 refs, todos usam)
    ↓
    Stage (46 refs, integração + perf/chaos via k6)
    ↓
    Produção (quando aprovado)
```

Problema central: QA e Stage acumulam **múltiplos papéis incompatíveis** no mesmo ambiente. O handbook descreve uma esteira linear (dev → QA → Stage → prod), mas a operação real é **hub-and-spoke em QA** com Stage como ambiente secundário sobrecarregado.

Evidência no código (performance-tests):

```
baseUrl = __ENV.ENV === 'qa'
  ? 'https://qa.youse.io/auto/order'
  : 'https://www-stage.youse.io/auto/order'
```

Testes de carga alternam entre os dois ambientes compartilhados — exatamente o comportamento que o TO-BE elimina.

4. Problemas do Dia a Dia (Catálogo)

Estes são os problemas **reais** relatados pelo time e **confirmados pela auditoria** — devem ser usados na conversa com liderança.

3.1 QA como "ambiente universal"

#	Problema	Quem é afetado	Impacto	Evidência auditoria
P1	Vários squads sobem versões diferentes no mesmo QA	QA, Produto	Ambiente quebrado ou inconsistente no momento do teste	313 refs QA em 461 repos
P2	Desenvolvedores testam features incompletas em QA	QA	Cenários bloqueados; dados corrompidos	<code>.helm/qa.yaml</code> em dezenas de serviços
P3	Produto valida fluxos enquanto dev ainda deploys	Produto, QA	Falso "bug" ou falsa "aprovação"	Deploy manual sem gate
P4	Mensagens no Slack: <i>"vou mexer no QA, parem de usar"</i>	Todos	Parada de trabalho; perda de produtividade	Sem calendário de ambientes
P5	Não há "versão oficial" do que está em QA	Release, QA	Impossível saber o que foi testado vs. o que vai para prod	Ratio QA:Stage = 6,8:1

3.2 Automação de testes instável

#	Problema	Causa	Impacto	Evidência auditoria
P6	Suite E2E roda em QA compartilhado	Ambiente muda durante execução	Falsos positivos; time ignora pipeline vermelho	<code>qa-e2e-tests-automation</code> , <code>automation-tests</code> → QA
P7	Colaboradores alteram regras de negócio em QA	Sem ambiente dedicado	Testes quebram sem mudança de código	Specs em <code>youse</code> , <code>notifier</code> , <code>order-service</code> apontam QA
P8	Feature flags / toggles desabilitados manualmente	Sem governança por ambiente	Automação falha em cascata	Não documentado no handbook
P9	Nomes de campos, labels ou fluxos alterados para teste manual	Dados e UI voláteis	Manutenção constante dos scripts	<code>platform-automated-testing/.env.qa</code>

3.3 Stage sobrecarregado

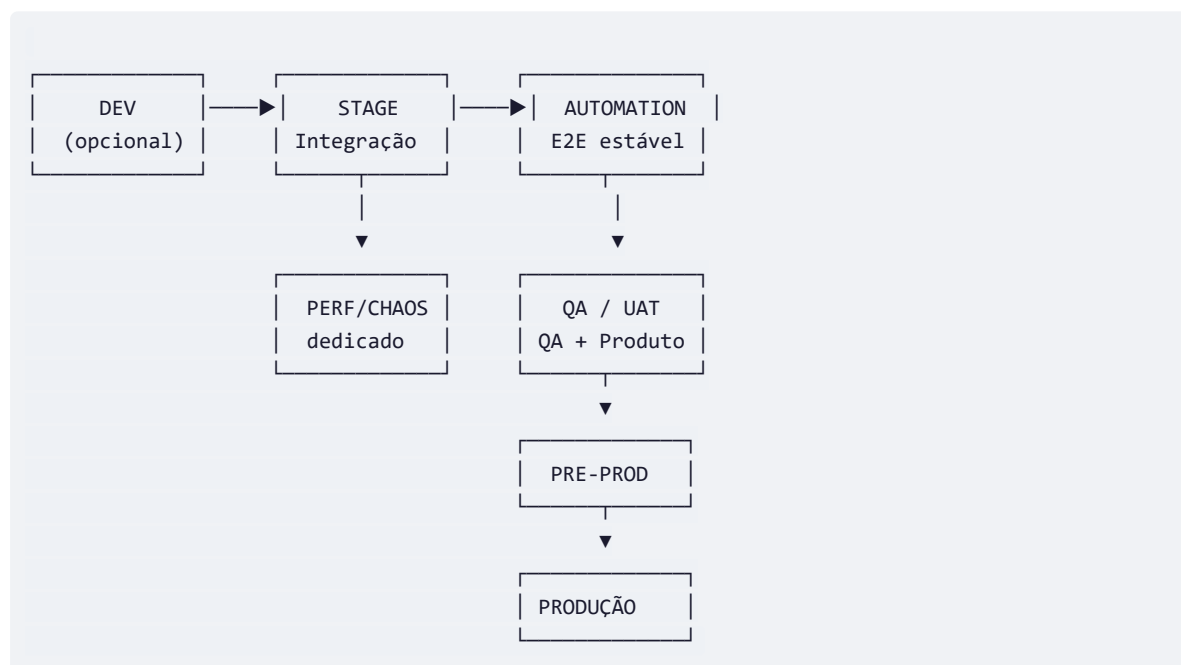
#	Problema	Causa	Impacto	Evidência auditoria
P10	Testes de performance rodam no Stage	Falta de ambiente dedicado	Stage lento ou indisponível para integração	performance-tests → Stage via k6
P11	Testes de caos (Chaos Engineering) no Stage	Mesmo ambiente de integração CI	Serviços derrubados; builds falham sem culpa do dev	Sem perf.youse.io (0 refs)
P12	Stage e QA com papéis sobrepostos	Estratégia não definida	Confusão sobre "onde validar o quê"	Handbook ≠ operação

3.4 Sintomas organizacionais

- Re-testes frequentes ("funcionou ontem, hoje quebrou")
- Dúvida se bug é de código ou de ambiente
- Releases atrasadas por falta de janela estável
- Automação vista como "frágil" em vez de rede de segurança
- Cultura reativa (Slack) em vez de processo preventivo (calendário + ambientes)
- Handbook desatualizado — novos devs aprendem processo errado
- 117 Jenkinsfiles + 159 CircleCI configs sem padronização de promote entre ambientes

5. Situação Desejada (TO-BE) — a estratégia completa

5.1 Camada 1 — Ambientes fixos (mapa)



5.2 Papéis de cada ambiente (linguagem simples)

Ambiente	URL sugerida	Analogia	Quem usa	Pode quebrar?
DEV	dev.youse.io ou preview por PR	Mesa de rascunho	Desenvolvedores	Sim, sempre
STAGE	www-stage.youse.io (existente)	Linha de montagem	CI após merge	Ocasionalmente
AUTOMATION	auto.youse.io	Laboratório de robôs	Pipeline E2E/API	Não (controlado)
PERF/CHAOS	perf.youse.io	Pista de stress	SRE, performance	Sim, de propósito
QA / UAT	qa.youse.io (existente)	Sala de prova	QA + Produto	Raramente
PRE-PROD	preprod.youse.io	Ensaio geral	Release, smoke final	Quase nunca
PROD	www.youse.io	Show ao vivo	Clientes	Nunca

5.3 O que muda na prática?

Hoje	Depois
"Testo no QA"	"Testo no ambiente certo para minha atividade"
Slack: parem de usar QA	Calendário de mudanças + ambientes isolados
Automação no QA (qa-e2e-tests-automation → QA)	Automação no Automation (auto.youse.io)
Performance/caos no Stage/QA (performance-tests)	Performance/caos no Perf/Chaos (perf.youse.io)
QA instável (313 refs, uso universal)	QA recebe release candidate estável
Handbook descreve 3 ambientes	Handbook atualizado com 7 ambientes e papéis
159 CircleCI + 117 Jenkins sem promote padronizado	Pipeline unificado via deployment-orb
Dev e QA disputam o mesmo QA	Preview por branch — cada tarefa tem ambiente próprio

5.4 Camada 2 — Ambiente temporário por branch (Preview)

Esta camada veio da sugestão do time e complementa a Camada 1 — não substitui. Enquanto os ambientes fixos organizam o **caminho até produção**, o preview organiza o **trabalho do dia a dia** sem conflito no QA.

A ideia em linguagem simples

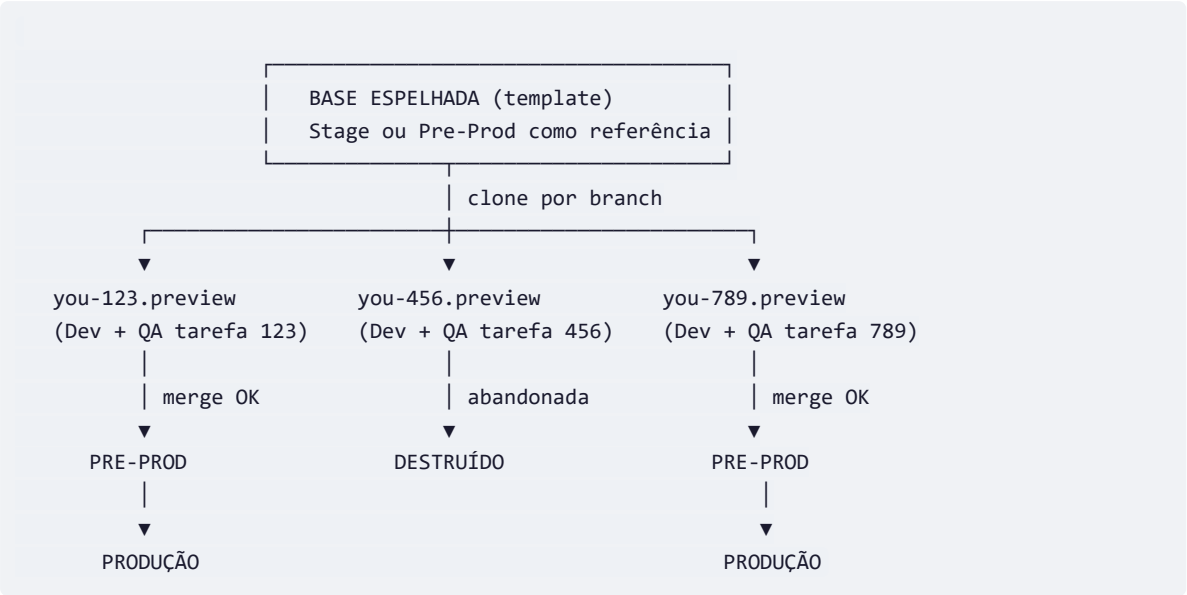
Imagine a produção como um **prédio modelo**. Para cada tarefa (branch), o pipeline **monta uma réplica temporária** desse prédio — só para aquela feature. O dev constrói dentro dela. Quando termina, avisa o QA. O QA testa **na mesma réplica** (mesma branch, mesma URL). Aprovado → merge → segue para Pre-Prod → Produção. Fim da tarefa → **demolição automática** da réplica.

Fluxo proposto

- 1. Dev abre branch feature/YOU-123-nova-cotacao
- 2. CI detecta branch → provisiona preview (namespace K8s + URL única)
Ex.: https://you-123.preview.youse.io
- 3. Dev desenvolve e testa NO PREVIEW (não no QA compartilhado)
- 4. Dev move card para "Pronto para QA" → notifica analista
- 5. QA acessa a MESMA URL do preview da branch YOU-123
- 6. QA aprova → merge na main
- 7. Pipeline promote: Stage → Automation → Pre-Prod → Prod
- 8. Preview YOU-123 é destruído automaticamente (merge ou fechamento da branch)

Importante: o QA **não precisa criar um clone separado**. Ele usa o **mesmo ambiente da branch** que o dev já subiu. Isso evita divergência ("no meu ambiente funciona").

Diagrama



Por que isso resolve o problema da Youse

Problema hoje	Como preview por branch resolve
Todos no QA	Cada tarefa tem URL própria — zero conflito
Slack "parem de usar QA"	Ninguém precisa parar — ambientes são independentes
QA encontra ambiente quebrado	QA testa exatamente o que o dev entregou na branch
Automação flaky no QA	E2E de regressão continua em auto.youse.io ; preview roda smoke da branch
Performance no Stage	Perf continua em perf.youse.io ; preview não roda carga

O que espelhar (base do clone)

Opção	Descrição	Quando usar
Espelho de Pre-Prod (recomendado)	Configs, toggles e integrações sandbox iguais à prod	Features que cruzam vários serviços
Espelho de Stage	Mais barato, já integrado	Features de um serviço só
Híbrido	Serviço alterado no preview + dependências apontam para Stage	Microserviços (maioria dos casos Youse)

Para a Youse, com muitos microserviços, o modelo mais realista é o **híbrido**: no preview sobe só o que mudou na branch; o resto continua apontando para Stage.

Como implementar (visão geral)

1. Dev abre branch → pipeline cria URL exclusiva (`you-123.preview.youse.io`)
2. Dev e QA usam essa URL durante a tarefa
3. Merge ou fechamento da branch → ambiente é removido automaticamente
4. Time de plataforma configura isso nos pipelines existentes (`deployment-orb` , CircleCI)

*Passo a passo técnico (DNS, Kubernetes, Helm) está no **Anexo D**.*

Regras importantes

Regra	Detalhe
Preview nunca substitui Pre-Prod	Pre-Prod continua obrigatório antes de prod
Preview não roda performance/caos	Usar <code>perf.youse.io</code>
Preview não é ambiente de automação de regressão	Regressão full em <code>auto.youse.io</code>
TTL máximo	72h sem atividade → destruição automática
Custo	Namespace + 1–3 pods por serviço alterado — muito menor que ambiente QA dedicado por squad

Piloto sugerido (30 dias)

1. Escolher **1 serviço** (`sales-frontend` ou `youse monolith`) + **1 squad**
2. Implementar preview só para esse repo
3. Medir: conflitos no QA, tempo de espera do QA, msgs Slack
4. Expandir para top 10 repos com mais deploys em QA

Relação com os outros ambientes

Ambiente	Papel	Efêmero?
Preview (branch)	Dev + QA por tarefa	✅ Sim — morre com a branch
Stage	Integração pós-merge	Não
Automation	E2E regressão estável	Não
QA/UAT	Release candidate, produto	Não
Pre-Prod	Ensaio geral pré-prod	Não
Perf/Chaos	Carga e caos	Não

6. Fluxo de CI/CD e Promoção

5.1 Pipeline recomendado (com preview por branch)

```
0. Branch feature/* → preview efêmero sobe (dev testa isolado)
1. Dev finaliza      → QA testa no MESMO preview da branch
2. Merge main        → preview destruído → deploy automático STAGE
3. Stage verde       → promote AUTOMATION
4. E2E 100%          → candidato a QA/UAT (RC estável)
5. QA + Produto      → aprovação funcional do release
6. PRE-PROD          → smoke + security + performance leve
7. PROD              → deploy com aprovação + rollback plan
```

5.2 Gates (portões de qualidade)

Gate	Ambiente	Critério mínimo
G1	Stage	Build OK + testes unitários + integração
G2	Automation	Suite E2E/API ≥ 98% estável (7 dias)
G3	QA/UAT	Checklist funcional QA + sign-off Produto
G4	Pre-Prod	Smoke + scan segurança + SLA básico
G5	Prod	Aprovação release manager + janela de deploy

5.3 Regras de deploy

- Proibido deploy manual em QA, Automation e Pre-Prod** — somente pipeline.
- Stage** recebe todo merge na branch acordada (`main` / `develop`).
- Hotfix** segue pipeline acelerado, mas **nunca pula Pre-Prod** em mudanças críticas.
- Cada deploy registra: commit, autor, squad, versão — visível em dashboard.
- `deployment-orb` deve ser estendido com jobs de promote Stage → Automation → QA → Pre-Prod.
- Repositórios com `.helm/qa.yaml` devem ganhar `.helm/automation.yaml`, `.helm/preprod.yaml` conforme fases do roadmap.

7. Ambiente de Automação (solução para P6–P9)

6.1 Princípios

Princípio	Descrição
Isolamento	Nenhum teste manual ou experimento de produto
Imutabilidade relativa	Atualiza só quando Stage está estável
Dados controlados	Seed versionado; reset periódico
Toggles congelados	Snapshot documentado por versão

6.2 Governança de Feature Flags

Feature Flag Registry		
Toggle	Automation	QA
nova_cotacao	ON (fixo)	ON/OFF flexível
checkout_v2	ON (fixo)	gradual

Regras:

1. Alterar toggle em **Automation** exige PR + review QA Automation.
2. Teste E2E **nunca assume** estado implícito de toggle — lê config ou usa API de setup.
3. Mudança de regra de negócio que afeta automação: **primeiro** atualizar testes + registry, **depois** promover código.
4. QA pode experimentar toggles; Automation **não**.
5. Migrar `qa-e2e-tests-automation` e `platform-automated-testing` para target `auto.youse.io`.

6.3 Dados de teste

- Contas e CPFs de teste documentados em repositório interno (`/test-data` ou Confluence).
- Scripts de seed idempotentes executados no deploy para Automation.
- Proibido usar dados de produção anonimizados sem processo LGPD.
- Substituir `.env.qa` hardcoded em `platform-automated-testing` e `mob-flutter` por configs versionadas por ambiente.

8. Performance e Caos — Ambiente Dedicado (solução para P10–P11)

7.1 Problema atual

Testes de carga, stress e caos no **Stage** (e alternância com QA via `performance-tests`) derrubam serviços usados por:

- Integração contínua pós-merge
- Desenvolvedores validando integrações
- Outros squads em janela de teste

Evidência: `performance-tests/src/services/web-app/index.js` alterna entre `qa.youse.io` e `www-stage.youse.io` conforme variável de ambiente — **zero referências a `perf.youse.io`**.

7.2 Solução: ambiente `perf.youse.io`

Aspecto	Política
Propósito	Load, stress, soak, chaos experiments
Agendamento	Janelas fixas (ex.: ter/qui 22h–6h) + calendário compartilhado
Dados	Volume sintético; nunca compartilhar DB com QA
Chaos	Hypothesis documentada; rollback automático
Stage	Proibido testes destrutivos de perf/chaos
Migração	Atualizar <code>performance-tests</code> para target exclusivo <code>perf.youse.io</code>

7.3 Níveis de teste de performance

Nível	Onde	Quando
Smoke perf (< 5 min)	Stage	Todo merge (APIs críticas)
Carga média	Perf	Nightly
Stress / Caos	Perf	Agendado + aprovado SRE

9. Comunicação e Calendário (substituir Slack reativo — P4)

8.1 Fim das mensagens "parem de usar QA"

Substituir por:

1. **Calendário de Ambientes** (Google Calendar / Confluence / Datadog):

- Deploy planejado em QA
- Reset de dados
- Janela perf/chaos
- Freeze de release

2. **Canal #ambientes-youse** (Slack):

- Bot posta automaticamente início/fim de deploy
- Status page interna: ● Stage | ● QA | ● Automation em manutenção

3. **Política de freeze:**

- 48h antes de release major: QA congelado para RC aprovado
- Exceções só com aprovação Release Manager

8.2 RACI simplificado

Atividade	Dev	QA	Produto	DevOps	CTO
Deploy Stage	R	I	—	A	I
Promover Automation	C	R	—	A	I
Testes UAT em QA	C	R	R	I	—
Perf/Chaos	C	I	—	R/A	I
Aprovar Pre-Prod → Prod	C	C	C	R	A

R = Responsável, A = Aprovador, C = Consultado, I = Informado

10. Investimento e ROI

9.1 Investimento estimado

Item	Esforço	Custo infra (ordem de grandeza)
Ambiente Automation	2–3 sprints	30–50% de um ambiente Stage
Ambiente Perf/Chaos	1–2 sprints	40–60% (picos sob demanda)
Pre-Prod	2 sprints	50–70% de prod (escala reduzida)
Governança flags + dados	1 sprint	Baixo (processo + tooling existente)
Dashboard + calendário	0,5 sprint	Baixo
Atualização handbook + 461 repos (refs)	1–2 sprints	Baixo (refatoração configs)
Extensão deployment-orb	1 sprint	Baixo
Preview por branch (piloto)	1–2 sprints	Baixo — namespaces K8s sob demanda

9.2 Retorno esperado

Métrica	Baseline (estimado)	Meta 6 meses
Horas/semana perdidas com ambiente instável	~15–25h (time combinado)	-70%
Taxa de falso positivo automação	Alta	< 5%
Re-testes por release	Frequentes	-50%
Deploys que quebram QA	Comum	Raro
Mensagens "parem de usar QA"	Semanal	Zero (substituídas por calendário)
Refs QA no código (313)	Baseline	Redistribuídas por ambiente

Argumento para o CTO: o custo de 2–3 ambientes adicionais é **menor** que o custo oculto de dezenas de horas/semana de retrabalho e releases atrasados. A auditoria de 461 repos prova que o problema é sistêmico — não pontual.

11. Roadmap de Implementação (90 dias)

Fase 1 — Quick wins (Semanas 1–4) | Sem custo infra significativo

- [] Publicar este documento v1.1 e alinhar com squads
- [] Definir papéis oficiais Stage vs QA (comunicado + onboarding)
- [] Bloquear deploy manual em QA (somente pipeline)
- [] Criar calendário de ambientes e canal #ambientes-youse
- [] Separar config de feature flags por ambiente
- [] **Proibir perf/chaos destrutivo no Stage** — migrar para janela temporária acordada
- [] Atualizar `handbook/docs/v1/quality/ambientes.md` com gap analysis
- [] Inventariar 313 refs QA e classificar por tipo (deploy, test, config, infra)

Fase 2 — Preview por branch + Automation (Semanas 5–8)

- [] DNS wildcard `*.preview.youse.io` (Terraform + Route53)
- [] Estender `deployment-orb:jobs deploy-preview` e `destroy-preview`
- [] Piloto preview em 1 repo (`sales-frontend` ou `youse`)
- [] Integração Jira/Slack: URL do preview no card
- [] Provisionar `auto.youse.io` (Terraform + Helm)
- [] Migrar `qa-e2e-tests-automation` para Automation
- [] Implementar gate: Stage → Automation → QA
- [] Estender `deployment-orb` com job `promote-to-automation`

Fase 3 — Perf/Chaos + Pre-Prod (Semanas 9–12)

- [] Provisionar `perf.youse.io`
- [] Migrar `performance-tests` (k6) para target exclusivo Perf
- [] Provisionar Pre-Prod espelhando prod
- [] Checklist de release obrigatório
- [] Dashboard de saúde por ambiente
- [] Desativar legado `automation-tests` (Cucumber, 2021)

Fase 4 — Evolução contínua

- [] Expandir preview para top 10 repos (microserviços híbridos)
 - [] Contract testing entre microserviços
 - [] Métricas DORA por ambiente
 - [] Consolidar 117 Jenkinsfiles → CircleCI/deployment-orb
-

12. Regras do Dia a Dia (cartilha por perfil)

Para Desenvolvedores

Faça	Não faça
Teste feature em andamento	Preview da branch (<code>you-123.preview.youse.io</code>)
Teste integração pós-merge	Stage
Valide integração em Stage após merge	Mudar toggles em Automation
Aguarde promote para QA testar sua feature	Rodar testes de carga em Stage
Consulte refs de ambiente no seu repo	Assumir handbook está atualizado

Para QA

Faça	Não faça
Teste manual de tarefa em andamento	Preview da branch (mesma URL do dev)
Teste de release candidate	QA/UAT (RC promovido por pipeline)
Reporte instabilidade com hash de versão	Alterar dados globais sem aviso
Participe da governança de toggles Automation	Ignorar pipeline vermelho de E2E
Valide promote Automation → QA antes de UAT	Rodar E2E em QA compartilhado

Para Produto

Faça	Não faça
Agende UAT após aviso de RC em QA	Validar feature em QA enquanto dev deploys
Use checklist de aceite	Pedir "só sobe no QA rapidinho"
Consulte calendário de ambientes	Ignorar freeze de release

Para DevOps / SRE

Faça	Não faça
Mantenha dashboard de status	Perf/chaos no Stage
Automatize promote entre ambientes	Deploy manual sem registro
Atualize Terraform/Ansible por ambiente	Compartilhar DB entre QA e Perf
Estenda <code>deployment-orb</code> com novos targets	Manter 117 Jenkinsfiles divergentes

13. Riscos e Mitigações

Risco	Mitigação
"Muitos ambientes = caro"	Começar com Automation + política Stage; Pre-Prod fase 3
Resistência cultural	Quick wins visíveis; métricas antes/depois; auditoria como evidência
Ambientes divergem de prod	Pre-Prod espelha config prod; IaC compartilhado (Terraform)
Prazo de squads	Fase 1 não exige infra nova
313 refs QA para migrar	Inventário fase 1; migração incremental por repo
Legado Jenkins (117 repos)	Manter paralelo; migrar gradualmente para deployment-orb
Handbook desatualizado	Atualizar na fase 1; linkar este documento

14. Critérios de Sucesso do Projeto

O projeto será considerado **bem-sucedido** quando:

1. QA receber **apenas release candidates** promovidos por pipeline.
2. Automação rodar em ambiente **dedicado** (`auto.youse.io`) com taxa de flaky < 5%.
3. **Zero** testes de caos/destrutivos no Stage.
4. Calendário substituir avisos reativos no Slack.
5. Produto e QA confiarem que "aprovado em QA" = testado de verdade.
6. `performance-tests` apontar exclusivamente para `perf.youse.io`.
7. Handbook refletir a arquitetura real de ambientes.
8. Ratio refs QA:Stage reduzir de 6,8:1 para ~2:1 (QA + Automation + Perf absorvem papéis específicos).

Anexo A — Auditoria Completa dos Repositórios

Escopo: org `youse-seguradora` | **461 repos** | Auditor: `gabrielroquim-youse` | **Data:** 09/06/2026

A.1 Resumo executivo da auditoria

Dimensão	Resultado
Total repositórios	461
Ativos (updated 2025–2026)	~180
Arquivados / legado	~40 (ex.: <code>archived-repos</code>)
Com CI/CD configurado	~276 (CircleCI 159 + Jenkins 117)
Referências QA	313
Referências Stage	46
Referências Perf	0
Referências Automation	0

A.2 CI/CD — inventário

Ferramenta	Repos (approx.)	Repositórios representativos
CircleCI	~159	youse, sales-frontend, deployment-orb, billing-cloud, order-service
Jenkins	~117	youse-jenkins-v2, jose-scripts, serviços legados
GitHub Actions	~7	qa-e2e-tests-automation, repos recentes
Helm charts	~30+	youse (.helm/qa.yaml, .helm/stage.yaml), deployment-orb
Terraform	~15+	terraform, terraform-platform (paths qa/, stage/)
Ansible	~10+	devops (ansible_v2/vars/*/qa_envs.yaml)

Achados críticos CI/CD:

- Não existe job padronizado de **promote** entre ambientes no deployment-orb
- Deploy QA e Stage coexistem, mas QA concentra 6,8× mais referências
- Jenkins legado (117 repos) opera em paralelo ao CircleCI sem convergência
- GitHub Actions emergente apenas em automação (qa-e2e-tests-automation)

A.3 Automação — inventário

Repositório	Framework	Target atual	CI	Status
qa-e2e-tests-automation	Playwright	QA (qa.youse.io)	GitHub Actions	Ativo — migrar para Automation
automation-tests	Cucumber/Ruby	QA (features/support/env.rb)	—	Legado (2021) — desativar
platform-automated-testing	—	QA (.env.qa)	—	Ativo — migrar para Automation
youse	RSpec	QA (specs, .helm/qa.yaml)	CircleCI	Ativo — specs devem apontar Automation
performance-tests	k6	QA ou Stage (toggle)	—	Ativo — migrar para Perf

Achados críticos automação:

- 3 repos de automação apontam para QA compartilhado
- Nenhum repo aponta para ambiente dedicado (0 refs auto.youse.io)
- automation-tests (Cucumber) desatualizado desde 2021 — candidato a desativação
- Playwright (qa-e2e-tests-automation) é a suite moderna — prioridade de migração

A.4 Performance — inventário

Repositório	Ferramenta	Targets	Risco
performance-tests	k6	qa.youse.io / www-stage.youse.io	P10/P11 — alterna ambientes compartilhados

Código relevante:

```
// performance-tests/src/services/web-app/index.js
baseUrl = __ENV.ENV === 'qa'
  ? 'https://qa.youse.io/auto/order'
  : 'https://www-stage.youse.io/auto/order'
```

Achados críticos performance:

- Zero refs a `perf.youse.io` — ambiente não provisionado
- Testes de carga competem com integração e testes manuais
- Sem janela agendada documentada no código

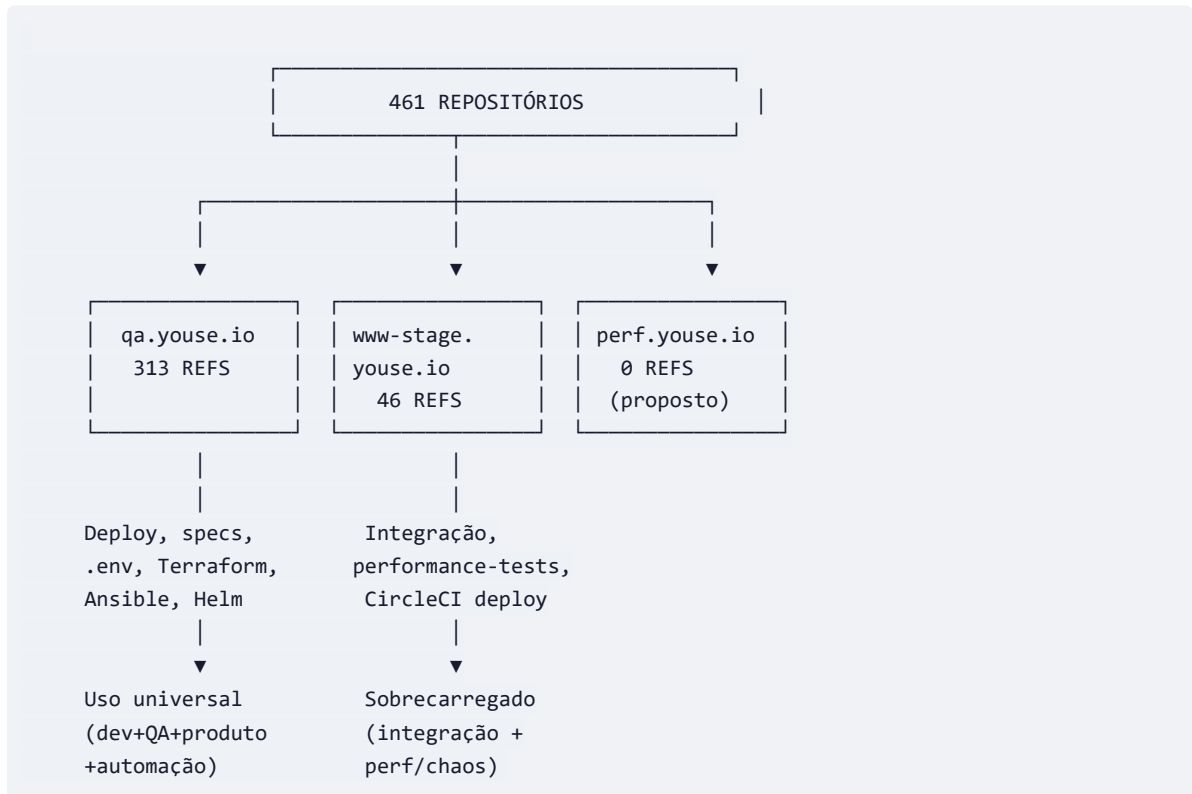
A.5 Apps — inventário (top repos por refs QA)

Repositório	Tipo	Refs QA	Deploy QA	Deploy Stage
youse	Core Rails	Alta	<code>.helm/qa.yaml</code>	<code>.helm/stage.yaml</code>
sales-frontend	Frontend React	Média	CircleCI	CircleCI
notifier	Microserviço	Média	<code>.env.example</code>	—
order-service	Microserviço	Média	specs QA	—
billing-cloud	Microserviço	Média	<code>.env.example</code>	—
inspection-service	Microserviço	Média	<code>.env.example</code>	—
bff	BFF	Média	docs QA	—
call-center-bff	BFF	Média	specs QA	—
mob-flutter	Mobile	Média	<code>.envs/.env.qa.json</code>	—
dashboard	Frontend	Baixa	<code>.env.example</code>	—
partner-api	API	Baixa	specs QA	—
c360	Serviço	Baixa	specs QA	—
devops	Infra	Alta	Ansible QA vars	—
terraform	IaC	Alta	paths <code>qa/</code>	paths <code>stage/</code>
terraform-platform	IaC	Alta	paths <code>qa/</code>	—

A.6 Docs — inventário

Repositório	Documento	Conteúdo	Gap
handbook	<code>docs/v1/quality/ambientes.md</code>	3 ambientes (QA, Stage, Prod)	Desatualizado — não reflete operação
insomnia-document	Collections	Requests apontando QA	Parcial
youse-sdk-ruby	<code>docs/inspection_service.md</code>	Refs QA	Parcial
bff	<code>engines/mobile_bff/docs/</code>	Refs QA	Parcial

A.7 Mapa de referências por ambiente



Distribuição estimada das 313 refs QA:

Tipo de referência	Approx. %	Exemplos
Config deploy (Helm, .env)	25%	.helm/qa.yaml , .env.qa
Specs de teste	30%	spec/*/_spec.rb com URLs QA
Infra (Terraform/Ansible)	20%	qa/ paths, qa_envs.yml
Documentação	10%	handbook, README, docs
Scripts/legado	15%	Jenkins, archived-repos

A.8 Próximas ações (pós-auditoria)

#	Ação	Responsável	Prazo	Prioridade
A1	Inventário detalhado das 313 refs QA (planilha por repo)	DevOps	Semana 1	P0
A2	Atualizar <code>handbook/docs/v1/quality/ambientes.md</code>	QA + DevOps	Semana 2	P0
A3	Estender <code>deployment-orb</code> com targets Automation/Perf/PreProd	DevOps	Semana 4–8	P0
A4	Migrar <code>qa-e2e-tests-automation</code> → <code>auto.youse.io</code>	QA Automation	Semana 5–8	P0
A5	Migrar <code>performance-tests</code> → <code>perf.youse.io</code>	SRE	Semana 9–12	P0
A6	Desativar <code>automation-tests</code> (Cucumber legado)	QA	Semana 8	P1
A7	Consolidar Jenkins → CircleCI (117 repos)	DevOps	Fase 4	P2
A8	Criar <code>.helm/automation.yaml</code> nos ~30 repos com Helm	DevOps	Semana 5–8	P1
A9	Provisionar infra Terraform para <code>auto.youse.io</code> e <code>perf.youse.io</code>	SRE	Semana 5–12	P0
A10	Dashboard de saúde por ambiente (Datadog/Confluence)	DevOps	Semana 9–12	P1

Anexo B — Referência de Infraestrutura (AWS)

Ambientes podem ser implementados como:

- **Namespaces Kubernetes** por ambiente (custo menor, isolamento lógico) — padrão atual em `gitops-kubernetes-addons` (`eks-compartilhado-qa`)
- **Contas AWS separadas** (isolamento forte — recomendado Pre-Prod/Prod)
- **Terraform/CloudFormation** com variáveis por ambiente — **mesmo código, configs diferentes**

Repositórios de infra existentes:

Repo	Escopo	Paths relevantes
<code>terraform</code>	Serviços individuais	<code>services//qa/</code> , <code>services/ /stage/</code>
<code>terraform-platform</code>	Plataforma compartilhada	<code>qa/business-management/</code> , <code>qa/it-management/</code>
<code>devops</code>	Ansible configs	<code>ansible_v2/vars/*/qa_envs.yml</code>
<code>gitops-kubernetes-addons</code>	K8s addons	<code>eks-compartilhado-qa/</code>

Variáveis por ambiente:

- URLs de APIs externas (sandbox vs prod)
- Connection strings (DB isolado por ambiente)
- Feature flag SDK keys
- Rate limits e scaling

- DNS (Route53): `qa.youse.io`, `www-stage.youse.io`, `auto.youse.io`, `perf.youse.io`, `preprod.youse.io`

Estimativa de custo mensal (ordem de grandeza):

Ambiente	% de Stage	Notas
Automation	30–50%	Escala fixa, baixo tráfego
Perf/Chaos	40–60%	Escala sob demanda (spot instances)
Pre-Prod	50–70%	Espelha prod em escala reduzida

Anexo C — Glossário

Termo	Significado
RC (Release Candidate)	Versão candidata a ir para produção
E2E	Teste ponta a ponta (simula usuário real)
Feature Flag / Toggle	Interruptor que liga/desliga funcionalidade sem deploy
Chaos Engineering	Testes que simulam falhas (servidor cai, rede lenta)
Gate	Portão de qualidade — só passa se critério OK
Flaky test	Teste que falha e passa sem mudança de código
UAT	Teste de aceitação pelo usuário de negócio / produto
Orb	Pacote reutilizável de config CircleCI (<code>deployment-orb</code>)
Helm	Gerenciador de charts Kubernetes (<code>.helm/qa.yaml</code>)
k6	Ferramenta de teste de carga (Grafana) usada em <code>performance-tests</code>
IaC	Infrastructure as Code (Terraform, Ansible)
Preview Environment	Ambiente efêmero por branch, destruído ao merge/fechar
TTL	Time-to-live — tempo máximo de vida de um preview antes de destruição automática

Anexo D — Implementação técnica do Preview por branch

Conteúdo para DevOps e engenharia de plataforma.

Peça existente na Youse	Papel no preview
<code>deployment-orb</code>	Jobs <code>deploy-preview</code> e <code>destroy-preview</code>
<code>gitops-kubernetes-addons</code>	Namespace por preview
<code>.helm/qa.yaml</code> , <code>.helm/stage.yaml</code>	Base para config de preview
<code>terraform-platform</code>	DNS <code>*.preview.youse.io</code>
CircleCI	Branch <code>feature/*</code> sobe preview; merge destrói

Passos resumidos: DNS wildcard → Helm por branch → namespace efêmero → TTL 72h → integração Jira/Slack com URL do preview.

